

1 Error Analysis

The Hospital 1 development run detects all 58 labelled erroneous invoices at the invoice-flag level. Table 1 summarises the overall flag-level result. Because the flag-level result is perfect on the development labels, this section does not list individual missed invoices. Instead, it analyses the systematic limitations that remain in category assignment, confidence, and exact corrected-total reconstruction.

Table 1: Overall flag-level performance on Hospital 1 development labels.

Metric	Value
True positives (TP)	58
False positives (FP)	0
False negatives (FN)	0
True negatives (TN)	855
Accuracy	1.000
Precision	1.000
Recall	1.000
F1	1.000

1.1 Per-category performance

Table 2 reports category-level detection. The overall flag is correct for every labelled erroneous invoice, but category-level recall is not perfect for every secondary category. In particular, `unknown_service` has recall 0.917: one invoice with a secondary `unknown_service` label was flagged under another dominant error category. This is a category-assignment limitation, not a missed invoice flag.

Table 2: Per-category detection on Hospital 1 development labels.

Category	Label invoices	Detected	Missed	Recall	Precision
<code>unknown_service</code>	12	11	1	0.917	1.000
<code>bundle_not_applied</code>	5	5	0	1.000	1.000
<code>contract_number_mismatch</code>	5	5	0	1.000	1.000
<code>cross_invoice_duplicate</code>	4	4	0	1.000	1.000
<code>daily_cap_exceeded</code>	4	4	0	1.000	1.000
<code>duplicate_invoice_id</code>	5	5	0	1.000	1.000
<code>exclusion_window_violation</code>	4	4	0	1.000	1.000
<code>invoice_total_mismatch</code>	6	6	0	1.000	1.000
<code>line_total_arithmetic</code>	6	6	0	1.000	1.000
<code>malformed_service_date</code>	6	6	0	1.000	1.000
<code>premium_incorrectly_applied</code>	6	6	0	1.000	1.000
<code>premium_omitted</code>	3	3	0	1.000	1.000
<code>service_date_after_invoice_date</code>	5	5	0	1.000	0.714
<code>service_date_out_of_window</code>	5	5	0	1.000	1.000
<code>unit_price_mismatch</code>	10	10	0	1.000	0.370
<code>volume_discount_incorrectly_applied</code>	4	4	0	1.000	0.800
<code>volume_discount_omitted</code>	4	4	0	1.000	1.000
<code>wrong_unit_basis</code>	11	11	0	1.000	0.917

1.2 Systematic failure modes

1.2.1 Ambiguous service matching

What goes wrong. Abbreviated or free-text billing descriptions cannot always be mapped uniquely to one contracted service. The system may flag the invoice correctly but still be unable to assign a single, defensible service identity.

Why it happens. The matcher relies primarily on textual and structural evidence. Short abbreviations, missing words, or generic clinical terms can leave more than one contract service structurally compatible.

Representative example. A description such as **Session Interm Hep Dial** has insufficient textual evidence to distinguish confidently between similar contracted services. Such cases are sent to manual review rather than forced into a match.

Mitigation. The matcher uses a conservative acceptance threshold, structural anchor checks, and a separate `needs_review` status. Billed price is used only as secondary evidence when the text and structure leave exactly one compatible candidate. Unmatched or ambiguous lines are flagged with low confidence rather than silently treated as correct.

1.2.2 Contract interpretation ambiguity

What goes wrong. Some contract clauses can reasonably be interpreted in more than one way. The system can detect that an invoice is inconsistent with the written rule, but the exact corrected total may depend on the chosen interpretation.

Why it happens. Section 5.1 states that a threshold premium is assessed against the aggregate daily quantity of the service delivered to the patient. It does not fully spell out whether the premium applies to all units that day or only to units above the threshold. Section 8 daily caps have a similar operational ambiguity when multiple lines for the same patient, service, and service day interact.

Representative example. For threshold premiums, the notebook applies the premium to all units once the daily aggregate threshold is crossed. This is a defensible reading of Section 5.1, but it is still an interpretation. For daily caps, four Hospital 1 labelled daily-cap cases do not reproduce the labelled corrected total when the written Section 8 cap is applied literally.

Mitigation. The notebook keeps the contract-faithful interpretation instead of hard-coding development labels. It reports the resulting residuals explicitly and documents the interpretation choice in the decision log.

1.2.3 Structural invoice-data ambiguity

What goes wrong. Some data-quality problems allow the system to say “this invoice is wrong” but do not allow it to say “the correct invoice total is exactly X ” without manual review.

Why it happens. Reused invoice identifiers, malformed service dates, and service dates after the invoice date can break the assumed one-to-one relationship between invoice records and line-item groups.

Representative example. INV-H1-000036 contains the malformed service date 2025-06-31. Reused invoice identifiers, such as those involving INV-H1-000152, require mapping line groups to invoice occurrences. The notebook assigns line groups to occurrences chronologically and validates the mapping against billed invoice totals where possible, but exact reconstruction may still require review.

Mitigation. The pipeline processes reused invoice IDs by transaction occurrence rather than merging all lines together. It validates occurrence mapping against billed totals, marks malformed dates as invalid, and lowers confidence when the contract basis cannot be established automatically.

1.2.4 Complex interacting pricing rules

What goes wrong. Bundles, facility multipliers, plan-tier multipliers, premiums, weekend uplifts, volume discounts, and daily caps can interact. Detecting that an error exists is easier than reconstructing the exact corrected total.

Why it happens. Clause 3.2 imposes a strict ordering: bundle $\rightarrow$ facility $\rightarrow$ plan $\rightarrow$ premium/uplift $\rightarrow$ volume discount $\rightarrow$ quantity. Rounding is applied after each step. A small interpretation error in one step propagates into the corrected total.

Representative example. INV-H1-000004 is labelled `unit_price_mismatch|volume_discount_incorrectly_a`. The invoice is flagged correctly, but the exact corrected total depends on whether the volume-discount threshold is applied before or after the line in question and on how the intermediate rounding is handled.

Mitigation. The pricing engine implements the clause ordering explicitly, uses integer cents and half-up rounding, and reports residuals by failure type rather than hiding them. When a defensible contract rate cannot be established, the billed amount is carried through provisionally and confidence is lowered.

1.3 Expected-total residuals

Exact corrected-total reconstruction is strong but not perfect on the development labels. Table 3 reports exact matches, and Table 4 reports the remaining systematic residuals.

Table 3: Exact `expected_total_cents` match on Hospital 1 development labels.

Subset	Exact match
Overall	99.5%
Labelled-correct invoices	100.0%
Labelled-erroneous invoices	91.4%

Table 4: Expected-total residuals by failure type.

Failure type	Invoices	Mean abs. diff. (cents)	Total abs. diff. (cents)
Daily-cap correction	4	76,118.75	304,475
Unknown-service provisional pricing	1	43,650.00	43,650

1.4 Interpretation

The Hospital 1 development run demonstrates strong flag-level detection. The remaining weaknesses are not primarily missed flags. They are systematic limitations in three areas:

1. mapping ambiguous billing descriptions to a unique contracted service;
2. resolving contract clauses that admit more than one reasonable reading;
3. reconstructing an exact corrected total when structural data problems or interacting pricing rules create uncertainty.

The confidence values are deterministic audit-policy scores, not calibrated probabilities. They express evidence quality and ambiguity. The system is designed to prefer a flagged uncertainty over a confident but unsupported answer. Hospital 1 labels were used for development evaluation and calibration; the runtime detector does not read the label file.